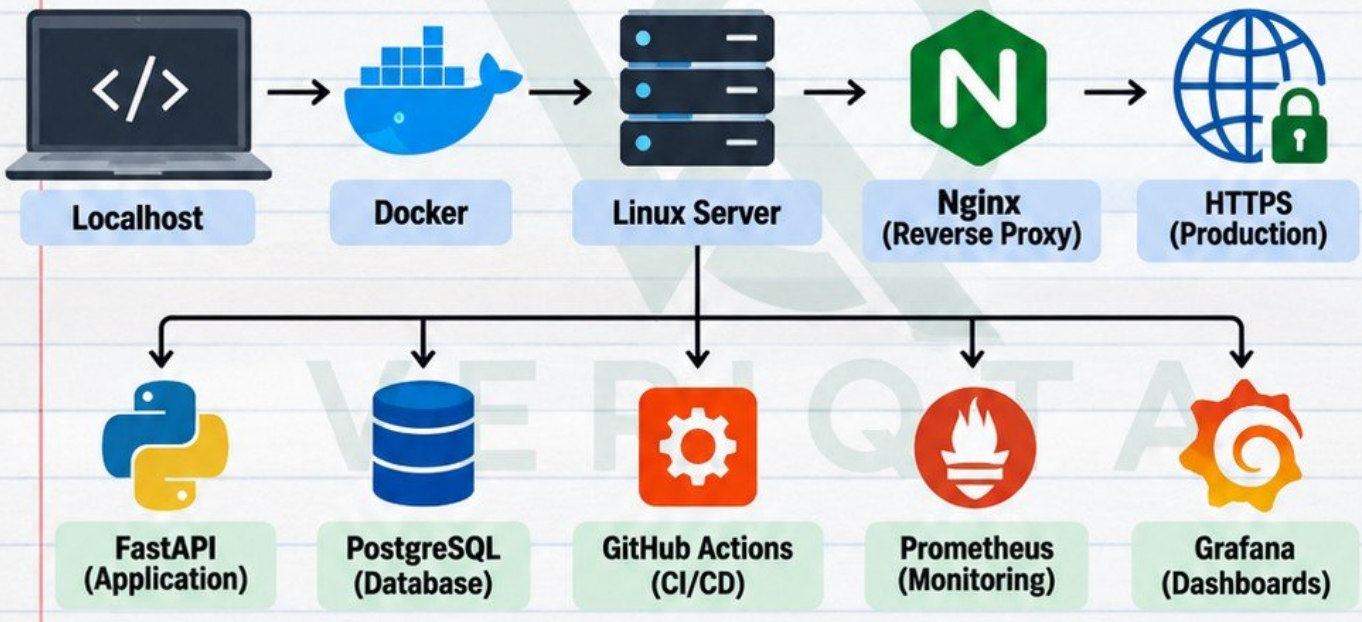
VERIQTA

From Localhost to Production

Deploy, Secure, Monitor, and Troubleshoot a Real App



- ✅ Build a real application
- ✅ Deploy to a Linux server
- ✅ Configure a domain and HTTPS
- ✅ Secure the application
- ✅ Set up CI/CD
- ✅ Monitor with real metrics and logs
- ✅ Troubleshoot common production issues
- ✅ Create backups and recovery steps
- ✅ Follow production best practices
- ✅ Gain practical DevOps and cloud skills

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

The Application We Are Taking to Production

VERIQTA

1. Meet OrderFlow

OrderFlow is a simple order-tracking API that allows users to create orders, retrieve orders and check the health of the service. We will take this same application from our laptop to a real server, secure it, monitor it and troubleshoot it.

Main Operations

Create an order `POST /orders`	Create a new order with item, quantity and price.
Retrieve an order `GET /orders/{id}`	Get the details of a specific order by ID.
Check application health `GET /health/live`	Verify that the application is running and ready.

2. How the Application Works



3. Running Process vs Working Application vs Ready for Traffic

Running Process

- The application process is running in memory.
- It is not enough to know if it actually works.
- Example: Uvicorn is running.

Working Application

- The application starts and responds to requests correctly.
- Endpoints return the expected data (e.g. /health/live works).
- It works for you, but not yet secure, reliable or production-ready.

Ready to Receive Traffic

- The application works and is properly configured.
- It is secured, monitored, can handle real users and has essential production setup (e.g. reverse proxy, HTTPS, logging, database, backups).

4. Practical: Check the Health Endpoint

```
curl -i http://127.0.0.1:8000/health/live
```

Example Output (Success)

```
HTTP/1.1 200 OK
content-type: application/json
{"status":"ok"}
```

5. What This Success Proves (and Does Not Prove)

What it proves

- The application process is running.
- The /health/live endpoint responds correctly.
- The application can handle a basic request.

What it does NOT prove

- The database is working.
- The application is secure.
- It can handle real user traffic.
- It is configured for production (e.g. HTTPS, monitoring).
- It is reliable or fault-tolerant.

Make Localhost Tell the Truth

VERIQTA

1. Run FastAPI Locally with Uvicorn

1. **Create and activate a virtual environment**

```
python -m venv venv
source venv/bin/activate  # Mac/Linux
# or
venv\Scripts\activate  # Windows
```

2. **Install dependencies**

```
pip install fastapi uvicorn
```

3. **Run the application**

```
uvicorn main:app --reload --host 127.0.0.1 --port 8000
```

2. What Does This Start?

This command starts the FastAPI application using Uvicorn, a web server for Python.

- **--reload:** automatically restarts when code changes (development only).
- **--host 127.0.0.1:** binds to your local machine only.
- **--port 8000:** application will be available on port 8000.

3. Understand 127.0.0.1, Port 8000 and HTTP

127.0.0.1 (Localhost)

- It refers to your own machine.
- Traffic never leaves your computer.
- Used for local development and testing.

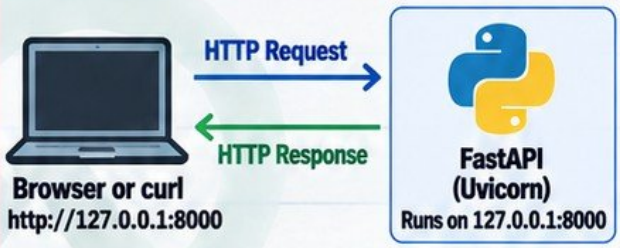
Port 8000

- A port is like a door for network traffic.
- Uvicorn listens on port 8000 by default.
- You can change it if needed.

HTTP Requests

- A way for clients (browser, curl, apps) to communicate with your application.
- Example: GET /health/live
- You send a request and receive a response.

4. What Happens When You Access It?



- You send an HTTP request to 127.0.0.1 on port 8000.
- Uvicorn receives the request and passes it to your FastAPI app.
- The app processes the request and returns a response (usually JSON).

5. Practical: Check the Health Endpoint

1. **Send a request using curl:**

```
curl -i http://127.0.0.1:8000/health/live
```

2. **Example output (success):**

```
HTTP/1.1 200 OK
content-type: application/json
date: Wed, 29 Oct 2026 10:30:00 GMT
server: uvicorn

{"status":"ok"}
```

6. What This Success Proves

- The application process is running.
- The endpoint /health/live is working.
- Your local environment and dependencies are correct.
- You can send and receive HTTP requests.

7. What This Does NOT Prove

- The database is working.
- The application is secure.
- It can handle real users or production traffic.
- It is configured for production (HTTPS, monitoring, etc).
- It will continue to work after a system restart.

8. Now Intentionally Stop the Server

1. In the terminal where Uvicorn is running, press **Ctrl + C**
2. Then send the request again:

```
curl -i http://127.0.0.1:8000/health/live
```

9. Example Output (Connection Failure)

```
curl: (7) Failed to connect to 127.0.0.1 port 8000:
Connection refused
```

This means there is no application listening on port 8000.
The server is stopped, so your request cannot be processed.

Turn Working Code Into a Reproducible Release

VERIQTA

1. Project Repository Structure

```
orderflow/
├── app/              # FastAPI application code
├── tests/            # Automated tests
├── requirements.txt  # Pinned dependencies
├── .env.example      # Example configuration
├── .gitignore        # Files to ignore in Git
├── README.md         # How to run the project
├── docker/           # (Next chapters)
└── scripts/          # Useful scripts
```

2. Pin Dependencies

Pinning dependencies ensures everyone installs the same library versions. This avoids "it works on my machine" problems.

requirements.txt

```
fastapi==0.115.0
uvicorn==0.30.6
pydantic==2.9.2
pytest==8.3.3
httpx==0.27.2
```

Why it matters:

- Prevents unexpected breaking changes.
- Improves stability.
- Makes the environment reproducible.

3. Use .gitignore

The `.gitignore` file tells Git which files and folders to ignore. This prevents accidental commits of sensitive or unnecessary files.

.gitignore (example)

```
.env
__pycache__/
*.pyc
.pytest_cache/
.venv/
.idea/
.vscode/
*.log
```

Why it matters:

- Keeps your repository clean.
- Prevents committing secrets.
- Reduces repository size.
- Avoids environment-specific files.

4. Introduce Automated Tests

Automated tests check that the application works as expected. They run the same way on every engineer's machine and in CI/CD later.

tests/test_health.py

```python
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health_live():
    response = client.get("/health/live")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}
```

5. Configuration vs Secrets

Configuration

- Non-sensitive settings.
- Examples: app name, port, log level.
- Can be stored in Git (e.g. .env.example).

Secrets

- Sensitive information.
- Examples: database password, API keys.
- Never commit to Git.
- Use environment variables or a secrets manager in production.

6. Why Reproducibility Matters



Another engineer must be able to:

- Clone the repository.
- Install the same dependencies.
- Run the application.
- Run the same tests.
- Get the same results.

This is important because:

- Teams work together.
- You cannot always rely on your laptop.
- It reduces environment-related issues.
- It makes deployments predictable and repeatable.

7. Practical: Clone, Install and Run the Tests

```
# 1. Clone the repository
git clone <your-repo-url>
cd orderflow
# 2. Create and activate a virtual environment
python -m venv .venv
source .venv/bin/activate    # Mac/Linux
# or .venv\Scripts\activate   # Windows
# 3. Install dependencies
pip install -r requirements.txt
# 4. Run the tests
pytest -v
```

8. Expected Output

```
============================= test session starts ===
tests/test_health.py::test_health_live PASSED       [100%]
============================== 1 passed in 0.42s ===
```

9. What This Proves (and Does Not Prove)

What it proves

- The code runs correctly.
- Dependencies are installed.
- The health endpoint works.
- Tests pass in a clean environment.

What it does NOT prove

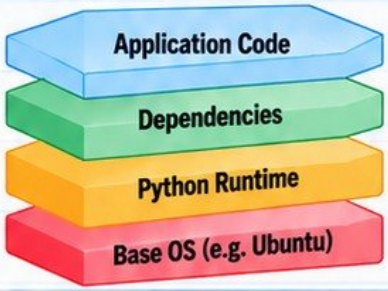
- The application is secure.
- It can handle real traffic.
- It is configured for production.
- The database and other services are working.

Package the Application Without Packaging Your Laptop

1. What is Docker?

- Docker packages your application and its dependencies into a container image.
- The image can run on any machine that has Docker, without relying on your laptop.
- This ensures a consistent and reproducible environment for development, testing and production.

2. Image Layers (How a Docker Image is Built)



→ Your application files (e.g. app/)
→ Installed packages (e.g. fastapi, uvicorn)
→ Python environment
→ Operating system

3. Dockerfile for OrderFlow

Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app/ ./app/
RUN useradd -m appuser
USER appuser
EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0",
    "--port", "8000"]
```

4. Important Concepts

Build Context	The folder you send to Docker when building the image (docker build .).
Image Layers	Each instruction in the Dockerfile creates a layer.
Container Process	The main command (CMD) runs as the main process inside the container.
Non-Root Execution	Run the application as a non-root user for better security.
.dockerignore	Specifies files and folders to exclude from the build context.

5. .dockerignore Example

.dockerignore

```
.git
__pycache__/
*.pyc
.env
.venv/
tests/
*.log
.idea/
.vscode/
```

Why it matters:

- Keeps the image small.
- Avoids copying sensitive files (e.g. .env).
- Speeds up the build process.
- Prevents unnecessary files from being included in the image.

6. Image vs Container

Docker Image

- A read-only template.
- Contains your application, dependencies and OS.
- Used to create containers.
- Example: orderflow:1.0

Docker Container

- A running instance of an image.
- Has its own process, file system and runtime environment.
- Can be started, stopped, restarted or removed.
- Example: orderflow:1.0 running on port 8000

7. Practical: Build and Run OrderFlow

```
# 1. Build the image
docker build -t orderflow:1.0 .

# 2. Run the container and map port 8000
docker run -d --name orderflow \
  -p 8000:8000 orderflow:1.0

# 3. Check running containers
docker ps

# 4. Test the application
curl -i http://127.0.0.1:8000/health/live
```

8. Inspect Logs When the Container Exits

```
# Check container logs
docker logs orderflow
# If the container has already stopped
docker logs orderflow --tail 50
# See the exit code
docker inspect orderflow --format '{{.State.ExitCode}}'
```

9. Common Issues and How to Fix Them

Issue / Error	Cause	Fix
Port already in use	Another process using 8000	Use a different port or stop the existing process
Container exits immediately	Application error	Check logs with docker logs
Module not found	Dependencies not installed	Check Dockerfile and rebuild
Permission denied	Running as non-root	Ensure correct file permissions
Cannot connect to localhost	Container not running	Check docker ps and logs

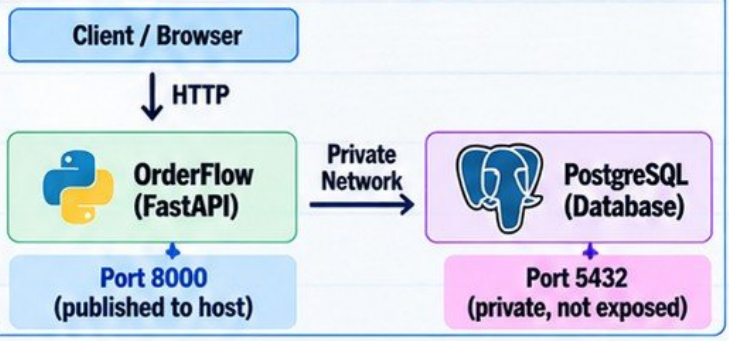
Give the Application Data That Survives a Restart

VERIQTA

1. Why We Need a Database

- Storing orders in application memory is temporary. Data is lost when the application restarts.
- A database like PostgreSQL stores data permanently on disk.
- This ensures order records are safe, reliable and available even after restarts, updates or failures.

2. Architecture with Docker Compose



3. Docker Compose Configuration

docker-compose.yml

```yaml
services:
  app:
    build: .
    container_name: orderflow
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=postgresql://orderflow:orderflow@db:5432/orderflow
    depends_on:
      - db

  db:
    image: postgres:16
    container_name: orderflow-db
    environment:
      - POSTGRES_USER=orderflow
      - POSTGRES_PASSWORD=orderflow
      - POSTGRES_DB=orderflow
    volumes:
      - postgres_data:/var/lib/postgresql/data
volumes:
  postgres_data:
```

4. Key Concepts

PostgreSQL	A reliable open-source database for storing application data.
Docker Compose	Runs multiple containers (app + database) together.
Private Container Networking	Services communicate securely using internal names (e.g. db).
Named Volumes	Stores database data on the host so it survives container restarts.
Database Migrations	Manages changes to the database schema in a controlled way.

5. Run the Application

```
# Start the containers
docker compose up -d

# Check running containers
docker compose ps

# View logs
docker compose logs -f
```

6. Test: Create and Retrieve an Order

```
# 1. Create a new order
curl -X POST http://127.0.0.1:8000/orders \
  -H "Content-Type: application/json" \
  -d '{"item":"Laptop","quantity":1,"price":1200}'

# 2. Retrieve the order (replace {id} with actual ID)
curl -X GET http://127.0.0.1:8000/orders/{id}

# 3. Check database directly (inside container)
docker compose exec db psql -U orderflow -d orderflow \
  -c "SELECT * FROM orders;"
```

7. Restart and Verify Persistence

```
# Restart the containers
docker compose down
docker compose up -d

# Retrieve the same order again
curl http://127.0.0.1:8000/orders/{id}
```

The order record is still there after the restart.

8. Common Issues and How to Fix Them

Issue / Error	Cause	Fix
Database connection failed	Wrong database URL or service name	Check DATABASE_URL and use db as the host
Data is lost after restart	No named volume	Use a named volume (e.g. postgres_data)
Permission denied	Wrong user or password	Check environment variables
Cannot connect to database from host	Port not exposed (secure by design)	Use docker compose exec to access the database

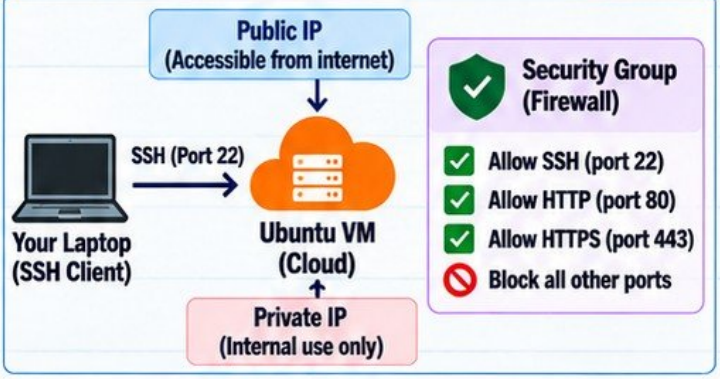
Provision a Server You Can Explain

VERIQTA

1. What We Are Doing

- Create an Ubuntu cloud virtual machine (VM).
- Understand public vs private IP addresses.
- Use SSH keys to connect securely.
- Configure security groups (firewall rules).
- Be aware of cloud costs.
- Understand the shared responsibility model.

2. High-Level Architecture



3. Public vs Private IP Address

Public IP Address

- Accessible from the internet.
- Used to connect to your server (e.g. via SSH).
- Changes if you stop and recreate the server (in most clouds).
- Keep it secure using security groups.

Private IP Address

- Used for internal communication inside the cloud network.
- Not accessible from the internet.
- Used between your server and other resources (e.g. database, internal services).
- More secure and recommended for internal traffic.

4. SSH Keys for Secure Access

- SSH keys allow you to connect without a password.
- More secure than passwords.
- You keep the private key on your laptop.
- Only the public key is added to the server.

```
# Generate a new SSH key (if you don't have one)
ssh-keygen -t ed25519 -C "your_email@example.com"

# The public key is usually here:
cat ~/.ssh/id_ed25519.pub
```

5. Create an Ubuntu Cloud VM (Example: AWS)

1. Choose Ubuntu (e.g. 22.04 LTS)
2. Select an instance type (e.g. t3.micro)
3. Add your SSH public key
4. Confi gure security group (allow port 22, 80, 443)
5. Launch the instance

Other Cloud Providers

- AWS (EC2)
- DigitalOcean
- Google Cloud (Compute Engine)
- Azure (Virtual Machine)
- Linode (Akamai)

6. Understand Cloud Costs

- You pay for the resources you use (compute, storage, network, etc).
- Small instances (e.g. t3.micro) are low cost and good for learning.
- Stop the server when not in use to avoid unnecessary charges.
- Monitor your usage and set up billing alerts.

7. Shared Responsibility Model

Cloud Provider Responsible For:

- Physical infrastructure
- Network, power, hardware
- Cloud platform availablity

You Are Responsible For:

- Operating system configuration
- Application security
- Access control (SSH keys, firewall)
- Keeping software updated
- Your data and backups

8. Practical: Connect Using SSH

```
# Connect to your server (replace with your public IP)
ssh -i ~/.ssh/id_ed25519 ubuntu@YOUR_PUBLIC_IP

# After connecting, you should see something like:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux x86_64)
```

9. Identify the Operating System

```
# Check OS information
cat /etc/os-release
# Example output (Ubuntu)
PRETTY_NAME="Ubuntu 22.04.5 LTS"
NAME="Ubuntu"
VERSION_ID="22.04"
```

10. Inspect CPU, Memory and Disk

```
# CPU information
lscpu | grep "Model name"
# Memory usage
free -h
# Disk usage
df -h
```

Prepare Linux Before Deploying Anything

VERIQTA

1. Create a Dedicated Deployment Account

- Do not use root for daily work.
- Create a dedicated user for deployment
- Give only the necessary permissions.
- Keep the account separate from your personal user.

```
# 1. Create a new user
sudo adduser deploy

# 2. Add to sudo group (only when needed)
sudo usermod -aG sudo deploy
```

2. Use SSH Key Authentication

- More secure than passwords.
- Use a key pair (private key on your laptop, public key on server).
- Disable password login later.

```
# Generate a key pair (on your laptop)
ssh-keygen -t ed25519 -C "deploy@example.com"

# Copy the public key to the server
ssh-copy-id -i ~/.ssh/id_ed25519.pub
deploy@YOUR_SERVER_IP

# Test login
ssh deploy@YOUR_SERVER_IP
```

3. Restrict Administrative Access

- Limit who can use sudo.
- Do not give full root access unnecessarily.
- Use sudo for specific administrative tasks.

Check sudo access:

```
ssh deploy@YOUR_SERVER_IP
sudo whoami
sudo -l
```

4. Keep the System Updated

- Install security updates regularly.
- Keep the operating system and packages up to date.
- This fixes vulnerabilities and improves stability.

```
# Update package list
sudo apt update

# Upgrade installed packages
sudo apt upgrade -y

# Remove unnecessary packages
sudo apt autoremove -y
```

5. Configure Firewall Rules

- Allow only the necessary ports (e.g. SSH, HTTP, HTTPS).
- Block all other inbound traffic.
- Use ufw (simple firewall for Ubuntu).

```
# Allow SSH (port 22)
sudo ufw allow 22

# Allow HTTP and HTTPS
sudo ufw allow 80
sudo ufw allow 443
# Check status
sudo ufw status
```

6. Why Not Run the Application as Root?

- If the application is compromised, attackers get full control.
- It can damage your system, files and other services.
- Use a non-root user inside containers and on the server.

Use non-root user in Docker (example):

```
# In Dockerfile
RUN useradd -m appuser
USER appuser
```

7. Practical: Verify SSH Access with the Deployment Account

```
# Connect using the deployment account
ssh deploy@YOUR_SERVER_IP

# Check who you are
whoami

# Check sudo access
sudo whoami

# Check shell and home directory
echo $SHELL
pwd
```

Expected output:

```
deploy
root
/bin/bash
/home/deploy
```

If connection fails:

- Check your key file and permissions (chmod 600 ~/.ssh/id_ed25519)
- Confirm the correct username and IP.
- Make sure port 22 is allowed in the firewall and security group.

8. Preserve a Recovery Path

- Before making major changes, ensure you have another way to access the server (e.g. cloud console or another admin user).
- Do not lock yourself out by disabling password login before confirming key-based access works.

Example: Disable password login (only after testing)

```
# Edit SSH configuration
sudo nano /etc/ssh/sshd_config
# Set this line (after confirming key access works)
PasswordAuthentication no
# Restart SSH service
sudo systemctl restart ssh
```

9. Summary: A Secure and Ready Server for Deployment

- ✅ Dedicated deployment account created.
- ✅ SSH key authentication configured.
- ✅ Password login restricted (after testing).
- ✅ Security updates installed.
- ✅ Firewall rules configured.
- ✅ Application will not run as root.
- ✅ Recovery path confirmed.

Key Takeaway:
A properly prepared Linux server is more secure, stable and ready for deploying real applications.

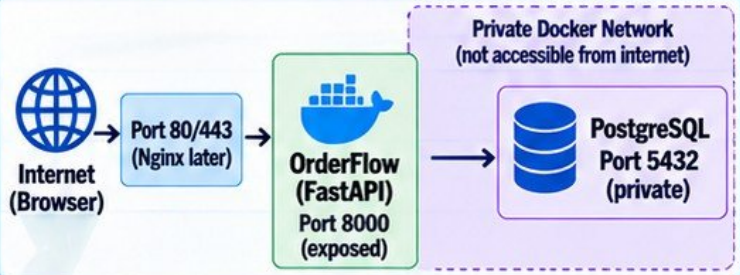
Deploy the First Version and Prove It Runs

VERIQTA

1. What We Are Doing

- Install the container runtime (Docker).
- Deploy the tested image using Docker Compose.
- Configure runtime environment variables.
- Start the application and keep the database private.
- Verify that it runs correctly.

2. Architecture on the Server



3. Install the Container Runtime

```
# Update system packages
sudo apt update && sudo apt upgrade -y

# Install Docker
sudo apt install -y docker.io

# Start and enable Docker
sudo systemctl enable --now docker

# Verify installation
docker --version
```

4. Deploy with Docker Compose

docker-compose.yml (example)

```
version: '3.9'
services:
  app:
    image: veriqta/orderflow:1.0
    container_name: orderflow-app
    ports:
      - "8000:8000"
    env_file:
      - .env
    depends_on:
      - db
  db:
    image: postgres:16
    container_name: orderflow-db
    environment:
      - POSTGRES_DB=orderflow
      - POSTGRES_USER=orderflow
      - POSTGRES_PASSWORD=strongpassword
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - app-network
networks:
  app-network:
    driver: bridge
volumes:
  postgres_data:
```

5. Environment Variables

.env (example)

```
APP_ENV=production
APP_PORT=8000
DATABASE_URL=postgresql://
orderflow:strongpassword@
db:5432/orderflow
SECRET_KEY=your-secret-key
LOG_LEVEL=info
```

6. Start the Application

```
# Start in detached mode
docker compose up -d

# Check running containers
docker compose ps
```

7. Inspect Running Containers

```
$ docker compose ps
NAME          IMAGE                 STATUS          PORTS
orderflow-app veriqta/orderflow:1.0 Up 5 seconds    0.0.0.0:8000->8000/tcp
orderflow-db  postgres:16           Up 5 seconds    5432/tcp
```

8. Check Application Logs

```
# View application logs
docker compose logs -f app

# View database logs
docker compose logs -f db
```

9. Test the Health Endpoint

```
# Check health response
curl -i http://127.0.0.1:8000/health/live
```

```
HTTP/1.1 200 OK
content-type: application/json

{"status":"ok"}
```

10. A Healthy Container Can Still Have a Broken App

A container can be "Up" but the application inside may still be failing.

Container is running

```
$ docker compose ps
orderflow-app    Up
```

Application is broken

```
$ curl -i http://127.0.0.1:8000/health/live
HTTP/1.1 500 Internal Server Error
{
   "detail":"Database connection failed"
}
```

11. Keep the Database Private

- Do not expose the database port (5432) to the internet.
- Use Docker internal networking.
- Only the application container can access the database.
- This reduces the attack surface.

12. Practical Checklist

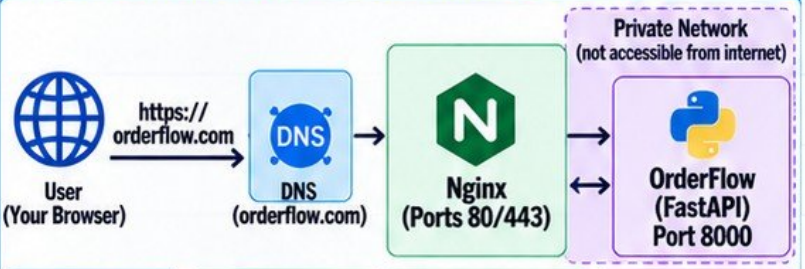
- [x] Docker installed and running.
- [x] Docker Compose started successfully.
- [x] Application container is running.
- [x] Database container is running (not publicly exposed).
- [x] Health endpoint returns 200 OK.
- [x] Application logs show no errors.
- [x] Verified that a container can be up even when the application fails.

Give the Service a Domain and a Safe Entry Point

1. What We Are Doing

- Connect a domain name to the server using DNS records.
- Explain ports 80 (HTTP) and 443 (HTTPS).
- Configure Nginx as a reverse proxy.
- Keep the application port 8000 inaccessible from the public internet.
- Trace a request from domain name to Nginx to the application.

2. Request Flow from Domain to Application



1. User enters domain name
2. DNS resolves to your server's public IP
3. Nginx receives request on port 80 or 443 (HTTPS)
4. Nginx forwards to application on internal port 8000

3. DNS Records

- Create an A record that points your domain (e.g. orderflow.com) to your server's public IP address.
- DNS translates the domain name to an IP address.
- DNS changes can take a few minutes to propagate.

Example (A record)

Type	Name	Value
A	orderflow.com	123.123.123.123

4. Understanding Ports 80 and 443

80 Port 80 (HTTP)

- Used for unencrypted web traffic.
- Requests and responses are in plain text.
- Usually redirected to HTTPS (port 443).

443 Port 443 (HTTPS)

- Used for encrypted web traffic.
- Secures data using TLS (SSL certificates).
- Recommended for production.

5. Nginx as a Reverse Proxy

- Nginx receives external requests on ports 80 and 443.
- Forwards requests to the application on port 8000 (internal only).
- Can handle SSL certificates, logging, and security controls.
- Hides the application from the public internet.

6. Nginx Configuration (Example)

/etc/nginx/sites-available/orderflow.com

```
server {
    listen 80;
    server_name orderflow.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto
    $scheme;
}
```

7. Keep Port 8000 Private

- Do not open port 8000 in your firewall or security group.
- Only Nginx on the same server (or internal network) should be able to access port 8000.
- This reduces the attack surface and keeps the application safe.

Firewall example (UFW)

```
# Allow HTTP and HTTPS only
sudo ufw allow 80
sudo ufw allow 443
# Do NOT allow port 8000
sudo ufw status
```

8. Trace a Request (Practical)

```
# 1. Check DNS resolution
nslookup orderflow.com
# 2. Test HTTP/HTTPS
curl -I http://orderflow.com
curl -I https://orderflow.com
# 3. Check Nginx is proxying
curl -I https://orderflow.com/
health/live
# 4. Check that port 8000
is NOT publicly accessible
curl -I http://orderflow.com:8000
# (should fail or timeout)
```

9. Common Misconfiguration (Wrong Upstream Port)

```
# Nginx config with wrong port (e.g. 9999)
location / {
    proxy_pass http://127.0.0.1:9999;
}
```

Result:

- Nginx returns 502 Bad Gateway.
- Application is healthy but Nginx cannot reach it.
- Check Nginx error logs and correct the upstream port.

10. Key Takeaways

- Domain name points to your server's public IP.
- Nginx handles external traffic on ports 80 and 443.
- Application runs on port 8000 (private).
- Use HTTPS in production.
- A correctly configured reverse proxy improves security, reliability and control.

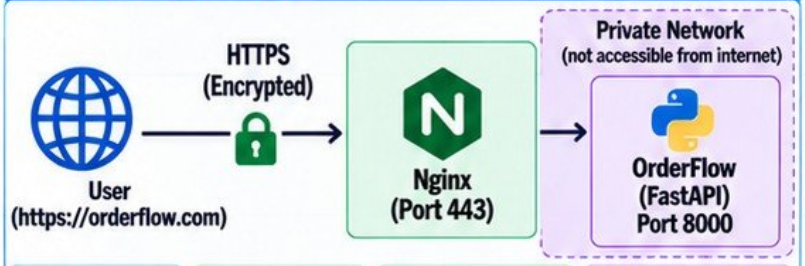
Enable HTTPS and Verify the Security Boundary

VERIQTA

1. What We Are Doing

- Issue a TLS certificate using Let's Encrypt.
- Configure HTTPS in Nginx.
- Test HTTP to HTTPS redirection.
- Verify certificate renewal.
- Understand encryption in transit.
- Know what HTTPS does and does not protect.

2. High-Level Flow



1. User connects using HTTPS (port 443).
2. Nginx terminates TLS (decrypts traffic).
3. Nginx forwards request to application on port 8000 (internal only).
4. Traffic between Nginx and application stays inside the private network.

3. Issue a Certificate with Let's Encrypt

```
# Install Certbot (Ubuntu)
sudo apt update
sudo apt install -y certbot python3-certbot-nginx

# Issue a certificate for your domain
sudo certbot --nginx -d orderflow.com -d www.orderflow.com
```

What this does:

- Requests a free TLS certificate from Let's Encrypt.
- Automatically configures Nginx to use the certificate.
- Sets up HTTP to HTTPS redirection.

4. Nginx HTTPS Configuration (Example)

```
server {
    listen 80;
    server_name orderflow.com www.orderflow.com;
    return 301 https://$host$request_uri;
}
server {
    listen 443 ssl http2;
    server_name orderflow.com www.orderflow.com;

    ssl_certificate /etc/letsencrypt/live/orderflow.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/orderflow.com/privkey.pem;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
}
```

5. Test HTTP to HTTPS Redirection

```
# Check that HTTP redirects to HTTPS
curl -I http://orderflow.com

HTTP/1.1 301 Moved Permanently
Location: https://orderflow.com/

# Check HTTPS response
curl -I https://orderflow.com

HTTP/2 200
content-type: application/json
```

6. Inspect the Certificate

```
# View certificate details
openssl s_client -connect
orderflow.com:443 -servername
orderflow.com

# Example output (shortened)
Certificate chain
  0 s:CN = orderflow.com
    i:C = US, O = Let's Encrypt
    ...
Verify return code: 0 (ok)
```

7. Verify Certificate Renewal

```
# Test renewal (dry run)
sudo certbot renew --dry-run

# Check renewal timer (systemd)
sudo systemctl status
certbot.timer
```

Automatic Renewal

- Let's Encrypt certificates are valid for 90 days.
- Certbot automatically renews them using a scheduled task.
- Always verify renewal is working.

8. Understanding Encryption in Transit

- HTTPS encrypts data between the user's browser and your server (Nginx) using TLS.
- It prevents attackers from reading or modifying the data in transit.
- It protects sensitive information such as login details, API requests, and responses.

9. What HTTPS Does Not Do

It does not secure your application code.

- It does not prevent authentication or authorization issues.
- It does not protect your database, internal network, or server from misconfigurations.
- You still need secure configurations, access controls, and proper coding practices.

10. Key Takeaways

- Use Let's Encrypt to enable HTTPS.
- Always redirect HTTP to HTTPS.
- Verify the certificate and renewal.
- HTTPS encrypts traffic in transit but does not make the entire application secure.
- Keep the application port 8000 inaccessible from the public internet.

Separate Configuration From Secrets

1. Key Concepts

- Runtime configuration vs secrets.
- Database credentials and other sensitive data.
- Environment files and environment variables.
- GitHub Actions secrets for CI/CD.
- Least privilege access.
- Accidental credential exposure and its risks.
- An environment variable is not a complete secrets-management solution.

2. Configuration vs Secrets

Configuration (Non-sensitive)

- Application settings (e.g. environment, log level).
- Used to control behaviour.
- Usually stored in environment variables or config files.
- Can often be committed to the repository (if non-sensitive).

Examples: APP_ENV, LOG_LEVEL, API_BASE_URL, PORT

Secrets (Sensitive)

- Credentials and secret keys (e.g. database password, API keys, JWT secret).
- Must not be committed to code.
- Should be stored in a secure secrets manager or CI/CD secrets.
- Use least privilege access.

Examples: DB_PASSWORD, SECRET_KEY, API_KEY, TLS_PRIVATE_KEY

3. Environment Files (.env)

```
# .env (example - do not commit to Git)
APP_ENV=production
PORT=8000
DATABASE_URL=postgresql://orderflow:********
LOG_LEVEL=info
SECRET_KEY=************************
```

- Add .env to .gitignore.
- Do not commit real secrets to the repository.
- Use example files (.env.example) with placeholder values.

4. .gitignore Example

```
# .gitignore (relevant section)
.env
.env.*
*.pem
*.key
*.crt
secrets/
*.log
__pycache__/
.DS_Store
```

5. GitHub Actions Secrets

- Store sensitive values in GitHub repository secrets.
- Use in workflows as environment variables.
- Never hardcode secrets in workflow files.
- Use least privilege (only the secrets required for the job).

```
# In GitHub Actions workflow
env:
  DATABASE_URL: ${{ secrets.DATABASE_URL }}
  SECRET_KEY: ${{ secrets.SECRET_KEY }}
```

6. Accidental Credential Exposure

- Committing secrets to GitHub.
- Printing secrets in logs.
- Sharing screenshots with credentials.
- Leaving default passwords.
- Exposing environment variables in error messages.
- Using overly permissive access (not least privilege).

7. Environment Variable vs Secrets Manager

Environment Variable

- Simple key-value pair.
- Stored in process or runtime environment.
- Suitable for basic configuration and non-highly sensitive values.
- Does not provide encryption, rotation, or access control.
- Not a complete solution for secret management.

Secrets Management Solution

- Designed to store, protect and manage secrets securely.
- Provides encryption, access control, and audit logs.
- Supports secret rotation.
- Examples: AWS Secrets Manager, HashiCorp Vault, Azure Key Vault.
- Recommended for production environments.

8. Practical: Rotate a Database Password

1. Generate a new strong password.
2. Update the password in the database.
3. Update the application configuration (environment variable or secrets manager).
4. Restart the application.
5. Verify that the application can connect using the new password.

9. Example: Update Database Password

```
# 1. Connect to the database
docker compose exec db psql -U orderflow
# 2. Update the password
ALTER USER orderflow WITH PASSWORD
'NewStrong@ssw0rd!';
# 3. Update environment variable (e.g. .env)
DATABASE_URL=postgresql://orderflow:NewStrong@ssw0rd!@db:5432/orderflow
# 4. Restart the application
docker compose up -d
# 5. Verify connectivity (from application logs or manually)
docker compose logs -f app
```

10. Key Takeaways

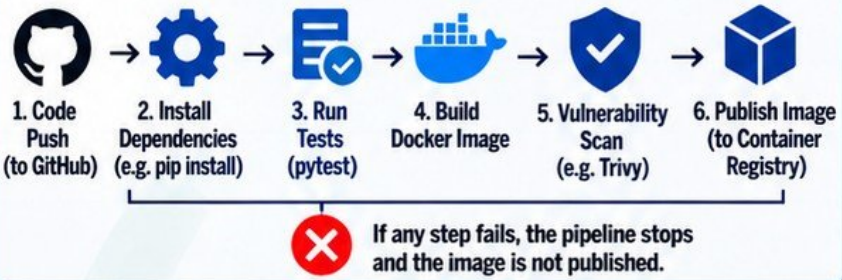
- Keep configuration and secrets separate.
- Do not commit secrets to Git.
- Use environment variables or a secrets manager.
- Apply least privilege access.
- Rotate credentials regularly.
- Be aware of accidental credential exposure.
- An environment variable is not a full secrets solution.

Build a CI Pipeline That Rejects Broken Code

1. What We Are Doing

- Create a GitHub Actions workflow.
- Install dependencies and run tests.
- Build a Docker image.
- Scan the image for vulnerabilities.
- Publish the image to a container registry (only if tests and scan pass).
- Understand what a green workflow means and what it does not mean.

2. CI Pipeline Flow



If any step fails, the pipeline stops and the image is not published.

3. GitHub Actions Workflow (Example)

.github/workflows/ci.yml

```yaml
name: CI
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  build-test-scan:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
    - name: Install dependencies
      run: |
        pip install -r requirements.txt
    - name: Run tests
      run: |
        pytest -v
    - name: Build Docker image
      run: |
        docker build -t orderflow:${{ github.sha }} .
    - name: Run vulnerability scan
      uses: aquasecurity/trivy-action@v0.24.0
      with:
        image-ref: 'orderflow:${{ github.sha }}'
        format: 'table'
        exit-code: '1'
        severity: 'HIGH,CRITICAL'
    - name: Login to container registry
      if: github.ref == 'refs/heads/main'
      uses: docker/login-action@v3
      with:
        username: ${{ secrets.REGISTRY_USERNAME }}
        password: ${{ secrets.REGISTRY_PASSWORD }}
    - name: Push image
      if: github.ref == 'refs/heads/main'
      run: |
        docker tag orderflow:${{ github.sha }} \
          ghcr.io/veriqta/orderflow:${{ github.sha }}
        docker push ghcr.io/veriqta/orderflow:${{ github.sha }}
```

4. Failing Test Blocks the Build

```
# Introduce a failing test (example)
# tests/test_order.py
def test_create_order():
    assert 1 == 2  # intentional failure
```

GitHub Actions Result (excerpt)

```
# Run pytest -v
================ FAILURES ================
test_create_order _____________________
E       assert 1 == 2

ERROR: 1 failed, 5 passed in 0.42s
Error: Process completed with exit code 1.
```

- Tests failed, so the pipeline stopped.
- The Docker image was not built or published.
- This prevents broken code from reaching production.

5. Vulnerability Scanning (Example with Trivy)

```
# Scan the built image
trivy image orderflow:latest
```

Vulnerability	Severity	Package	Version	Fixed
CVE-2024-1234	HIGH	openssl	3.0.7	3.0.9
CVE-2024-5678	CRITICAL	libcurl	7.88.1	8.2.0

- The scan checks for known vulnerabilities.
- You can fail the pipeline if HIGH or CRITICAL issues are found.
- This reduces the risk of deploying insecure images.

6. Publish to a Container Registry

```
# Example: Push to GitHub Container Registry
# (only if tests and scan pass)
docker tag orderflow:latest \
  ghcr.io/veriqta/orderflow:latest
docker push ghcr.io/veriqta/orderflow:latest
```

- Image is only published when all previous steps succeed.
- Allows you to deploy a tested and scanned image.
- Keeps your container registry clean and reliable.

7. Green Workflow ≠ Complete Safety

A green workflow means:

- Dependencies installed successfully.
- Tests passed.
- Image built successfully.
- Vulnerability scan passed (based on configured rules).
- Image published successfully.

It does NOT mean:

- Your application is free from all bugs.
- There are no logic errors.
- There are no runtime or configuration issues.
- There are no new vulnerabilities discovered tomorrow.
- Your infrastructure is fully secure.

8. Practical Steps

1. Make a small code change that causes a test to fail.
2. Commit and push to GitHub.
3. Watch the GitHub Actions workflow run and fail.
4. Fix the test.
5. Push again and verify that the workflow passes and the image is built (and published if on main).

9. Best Practices

- Run automated tests on every push.
- Fail the pipeline on critical vulnerabilities.
- Use least privilege for registry credentials.
- Keep secrets in GitHub Actions secrets.
- Build reproducible images (pin dependencies).
- Use clear and meaningful workflow logs.
- Monitor and review workflow results regularly.

10. Key Takeaways

- CI catches problems early.
- Automated tests, scans, and builds reduce risk.
- A green workflow is evidence of specific checks, not proof of complete safety.
- Never store secrets in code or workflow files.
- Use a container registry to store trusted images.
- A well-designed CI pipeline keeps broken and insecure code out of production.

Deploy a Version, Verify It, and Roll It Back

VERIQTA

1. What We Are Doing

- Deploy a new version using immutable image tags or digests.
- Verify the release with health and smoke tests.
- Record the previous version.
- Demonstrate rollback to the previous version.
- Understand why latest is a poor release identifier.
- Understand how database migrations can complicate rollback.

2. Deployment Flow



1. Build & Push (version tag)
2. Deploy new version on server
3. Verify (smoke test & health check)
4. If good keep it in production
5. If issues roll back to previous version

v1.0.0
v1.1.0
v1.2.0

- Use immutable tags (e.g. v1.2.0) or image digests.
- Do not use latest for releases.
- Always keep a record of the previously running version.
- Verify the application works as expected after deployment.

3. Deploy a Specific Version (Docker Compose)

```
# Deploy version v1.2.0
services:
  app:
    image: ghcr.io/veriqta/orderflow:v1.2.0
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=${DATABASE_URL}
    depends_on:
      - db
```

4. Verify the Release

```
# Check running container and version
docker compose ps

# Check application health endpoint
curl -I https://orderflow.com/health

# Smoke test (example endpoint)
curl https://orderflow.com/api/orders

# Check application logs
docker compose logs -f app
```

5. Record the Previous Version

```
# See previously built images
docker images | grep orderflow

# Tag the current production image
docker tag ghcr.io/veriqta/orderflow:v1.1.0 \
 ghcr.io/veriqta/orderflow:previous

# Or record the running version
docker compose ps
docker inspect orderflow-app \
    --format '{{.Config.Image}}'
```

6. Roll Back to the Previous Version

```
# Roll back to the previous known good version
services:
  app:
    image: ghcr.io/veriqta/orderflow:v1.1.0
    ports:
      - "8000:8000"
    environment:
      - DATABASE_URL=${DATABASE_URL}
    depends_on:
      - db
# Apply the changes
docker compose up -d
```

7. Why "latest" Is a Poor Identifier

- latest can change at any time.
- It is not specific to a tested release.
- Different environments may pull different images.
- It makes rollbacks difficult.
- It can lead to unexpected and uncontrolled deployments.

Use immutable tags or digests instead:

v1.2.0	→ specific version
v1.2.0-20261030	→ version with build date
sha256:abc123...	→ exact image digest

8. Database Migrations and Rollback

- Database migrations can change the data structure.
- Some changes are not easily reversible (e.g. dropping columns).
- Rolling back the application code may not be compatible with the new database schema.
- Use backward-compatible migrations when possible.
- Plan and test rollback procedures for both application and database.

9. Practical: Release, Detect a Defect, Roll Back

1. Build and push a new version with an intentional defect.
2. Deploy the new version to your server.
3. Run a smoke test and observe the failure.
4. Check logs to confirm the issue.
5. Roll back to the previous compatible version.
6. Verify that the application is working again.

10. Example: Detect a Failing Release

```
# Smoke test (returns error)
curl -I https://orderflow.com/health
HTTP/1.1 500 Internal Server Error
content-type: application/json

# Check logs
docker compose logs app --tail 20
ERROR  Unexpected value
ERROR  Failed to process order
```

11. Key Takeaways

- [x] Always deploy using immutable tags or digests.
- [x] Verify the release with health and smoke tests.
- [x] Keep a record of the previous version.
- [x] Avoid using latest for releases.
- [x] Database migrations can complicate rollback.
- [x] Have a tested rollback procedure.
- [x] A successful rollback restores service to the last known good state.

Read Logs Like an Engineer Investigating a Failure

VERIQTA

1. What We Are Doing

- Introduce structured logs.
- Understand request IDs and timestamps.
- Identify HTTP status codes.
- Find and interpret application exceptions.
- Learn about log retention.
- Use logs to reconstruct a failed request.
- Avoid logging personal data or credentials.

2. What Good Logs Contain

Timestamp — When it happened

Request ID — Correlate across services

Log Level — INFO, WARN, ERROR

HTTP Details — Method, path, status code

Message — Clear and structured information

```
{"timestamp":"2026-10-30T14:22:31.123Z", "level":"ERROR",
"request_id":"f3c2a1d9-8e7c-4b1a-9c2d-6d8e7f1a2b3c",
"method":"POST","path":"/api/orders","status_code":500,
"message":"Database connection failed","service":"orderflow"}
```

3. View Application Logs (Docker)

```
# View recent logs
docker compose logs -f app

# View last 100 lines
docker compose logs --tail 100 app

# Filter for errors
docker compose logs app | grep ERROR

# Follow logs in real time
docker compose logs -f --timestamps app
```

4. Understand Log Levels

Level	Meaning
DEBUG	Detailed information for troubleshooting.
INFO	Normal application behaviour.
WARN	Something unexpected happened but the app is still running.
ERROR	An operation failed.
CRITICAL	A serious problem that may stop the app.

5. Common HTTP Status Codes

Code	Meaning
200	Request successful.
201	Resource created.
400	Bad request (invalid input).
401	Unauthenticated (missing or invalid token).
403	Forbidden (no permission).
404	Resource not found.
500	Internal server error (application exception).
502	Bad gateway (upstream error).
503	Service unavailable.

6. Application Exception in Logs (Example)

```
2026-10-30T14:22:31.123Z ERROR [orderflow]
request_id=f3c2a1d9-8e7c-4b1a-9c2d-6d8e7f1a2b3c
POST /api/orders 500
Exception: psycopg2.OperationalError:
could not connect to server:
Connection refused
Is the server running on "db" (172.20.0.5)
and accepting TCP/IP connections on port 5432?
```

7. Reconstruct a Failed Request

1. Make a request and note the request ID from the response.
2. Search the logs using the request ID.
3. Follow the log entries from start to finish.
4. Identify the error and its cause.
5. Confirm the fix and retest.

8. Log Retention and Privacy

- Keep logs for a defined period (e.g. 7, 30, or 90 days).
- Store logs in a secure system (e.g. cloud logging service).
- Use log rotation to prevent unlimited disk growth.

- Do not log personal data (e.g. names, emails, phone numbers).
- Do not log credentials or secrets.
- Mask or redact sensitive information.
- Follow your organization's data protection requirements.

9. Practical: Trigger and Investigate a Failed Order

1. Send an order request with invalid data (e.g. missing product_id).
2. Observe the error response and note the request ID.
3. Find the related log entries on the server.
4. Identify the cause (e.g. validation error, database error).
5. Fix the issue and test again.

10. Example: Failed Order Request

```
# Send a request with invalid data
curl -X POST https://orderflow.com/api/orders \
 -H "Content-Type: application/json" \
 -d "{"product_id": "", "quantity": 1}"
```

Response (excerpt)

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
{"error":"product_id is required",
"request_id":"f3c2a1d9-8e7c-4b1a-9c2d-6d8e7f1a2b3c"}
```

11. Key Takeaways

- Logs show what happened and help you find the cause of a failure.
- Use structured logs with request IDs and timestamps.
- HTTP status codes tell you the result of a request.
- Logs help you reconstruct a failed request without exposing sensitive data.
- Set up log retention and rotation.
- Never log personal data or credentials.

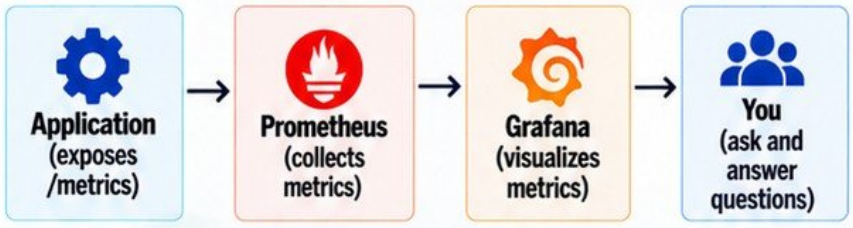
Build Monitoring That Answers Operational Questions

VERIQTA

1. What We Are Doing

- Introduce Prometheus metrics and Grafana dashboards.
- Monitor request rates, error rates, latency, CPU, memory and disk usage.
- Distinguish liveness, readiness and actual user experience.
- Build a small dashboard and generate test traffic.
- Observe changes in requests, errors and response time.
- Understand how monitoring answers real operational questions.

2. Monitoring Architecture (Simple Setup)



Application exposes metrics endpoint (e.g. :8000/metrics)

Prometheus scrapes metrics from your application and infrastructure.

Grafana creates dashboards with charts and alerts.

You use the data to understand system health and user experience.

3. Prometheus Metrics Endpoint (Example)

```
# Check metrics endpoint
curl http://localhost:8000/metrics

# HELP http_requests_total Total HTTP requests
# TYPE http_requests_total counter
http_requests_total{method="GET",status="200"} 1023
http_requests_total{method="GET",status="500"} 12

# HELP http_request_duration_seconds Request latency
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.1"} 240
http_request_duration_seconds_bucket{le="0.5"} 892
http_request_duration_seconds_bucket{le="1.0"} 1020
```

4. Example Grafana Dashboard



5. Key Metrics Explained

Metric	What It Means	Why It Matters
Request Rate	Number of requests per second	Shows usage and traffic patterns.
Error Rate	Percentage of failed requests (4xx, 5xx)	Shows reliability and problems.
Latency (Response Time)	Time taken to process a request	Affects user experience.
CPU Usage	How much CPU the system uses	Shows if the server is under heavy load.
Memory Usage	Amount of memory in use	Helps prevent out-of-memory issues.
Disk Usage	Storage consumption	Prevents disk full outages.

6. Liveness, Readiness and User Experience

Liveness

- Is the application process running?
- Simple health check (e.g. /health/live).
- Used to detect and restart a failed container.

Readiness

- Is the application ready to serve traffic?
- Checks dependencies (e.g. database).
- Used by load balancers and Kubernetes to route traffic.

User Experience

- Are real users able to complete their tasks?
- Measured by request rate, error rate, latency and business metrics (e.g. successful orders).
- This is what truly matters.

7. Generate Test Traffic (Example)

```
# Generate normal traffic (success)
for i in {1..100}; do
  curl -s http://localhost:8000/api/orders \
    -H "Content-Type: application/json" \
    -d '{"product_id":1,"quantity":1}' \
    -o /dev/null
done

# Generate some failing requests
for i in {1..20}; do
  curl -s http://localhost:8000/api/orders \
    -H "Content-Type: application/json" \
    -d '{"product_id":999,"quantity":1}' \
    -o /dev/null
done
```

8. Practical: Build and Observe

1. Configure Prometheus to scrape your application metrics.
2. Set up Grafana and create a dashboard.
3. Generate test traffic (normal and failing requests).
4. Observe changes in request rate, error rate and latency.
5. Check CPU, memory and disk usage during the test.
6. Answer specific operational questions from the dashboard.

9. Example Operational Questions

- Is the application running?
- Are users experiencing errors?
- How many requests are we handling per second?
- What is the current response time?
- Is the server running out of CPU, memory or disk?
- Are there unusual spikes in traffic or errors?
- Can users successfully complete an order?

10. Key Takeaways

- Monitoring turns real-time data into answers for operational questions.
- Metrics show the behaviour of your application and infrastructure.
- Liveness and readiness are not the same as user experience.
- A healthy container does not always mean a good user experience.
- Use dashboards and alerts to detect and investigate issues early.

Create Alerts That Lead to Useful Actions

1. Key Concepts

- **Alert thresholds:** when a metric crosses a defined value.
- **Duration:** condition must persist (e.g. 5 minutes) to avoid noise.
- **Severity:** classify alerts (warning, critical).
- **False positives:** alerts that fire but are not real issues.
- **Alert fatigue:** too many alerts reduce response time and trust.
- **Useful alerts:** must lead to a clear action and have the right level of detail.

2. Alerting Architecture (Overview)



- ✅ Prometheus detects when a condition is met.
- ✅ Alertmanager handles routing, grouping, deduplication and notifications.
- ✅ Alerts should include clear context and actionable information.
- ✅ Send alerts to channels your team actively monitors.

3. Example Alert Rules (Prometheus)

```yaml
# Application Unavailability
- alert: AppDown
  expr: up{job="orderflow"} == 0
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Orderflow application is down"
    description: "No response from orderflow
                  for more than 5 minutes."

# High HTTP 5xx Errors
- alert: HighHttp5xxErrors
  expr: rate(http_requests_total{job="orderflow",
  status=~"5.."}[5m]) > 0.05
  for: 5m
  labels:
    severity: warning
  annotations:
    summary: "High HTTP 5xx error rate"
    description: "More than 5% of requests are
                  failing in the last 5 minutes."

# Low Disk Space
- alert: LowDiskSpace
  expr: node_filesystem_avail_bytes{mountpoint="/"}
  / node_filesystem_size_bytes{mountpoint="/"} < 0.1
  for: 10m
  labels:
    severity: critical
  annotations:
    summary: "Low disk space on server"
    description: "Less than 10% disk space remaining
                  on {{ $labels.instance }}."
```

4. Alert Thresholds and Duration

Setting	Example	Why It Matters
Threshold	> 5% 5xx errors	Defines when the alert should fire.
Duration (for)	5 minutes	Prevents short spikes from triggering alerts.
Severity	warning / critical	Helps prioritise the response.
Labels	service, environment	Used for routing and grouping.
Annotations	summary, description	Provide clear context and action steps.

5. Example Alert Notification

Orderflow Alerts APP 10:24 AM

[CRITICAL] AppDown
Orderflow application is down
Instance: orderflow-1
Environment: production
Duration: 5m
Runbook: https://veriqta.com/runbooks
Action: Check container, logs and infrastructure.

[RESOLVED] AppDown
Orderflow application is back up
Instance: orderflow-1
Duration: 2m
Recovered at: 10:32 AM
Action: Verify service and monitor for recurrence.

6. Generate a Controlled Outage

```bash
# Stop the application to trigger alert
docker compose stop app

# or simulate high 5xx errors (example)
for i in {1..50}; do
 curl -X POST http://localhost:8000/api/orders \
   -d "{\"invalid\": true}"
done

# Fill disk space (test environment only)
sudo fallocate -l 500M /tmp/testfile
```

7. Verify Alert Recovery

```bash
# Start the application again
docker compose start app

# Check the alert status in Prometheus
# Alerts page
# http://localhost:9090/alerts

# Check that the alert is resolved
# in Alertmanager
# http://localhost:9093
```

8. Practical Steps

1. Set up Prometheus and Alertmanager.
2. Create the three alert rules (app down, 5xx errors, low disk).
3. Configure notification channel (e.g. Slack or email).
4. Trigger a controlled outage or error condition.
5. Confirm the alert fires with correct details.
6. Restore the service.
7. Verify that the alert is marked as resolved.

9. Common Issues and Solutions

Issue	Cause	Solution
Too many alerts	Threshold too low	Increase threshold or duration.
False positives	Short spikes	Use a longer for duration (e.g. 5m).
No alerts received	Notification misconfiguration	Check Alertmanager routes and channels.
Alert not resolved	Condition still true	Verify the service is healthy and metrics have recovered.

10. Key Takeaways

- ✅ Set meaningful thresholds and durations.
- ✅ Alert on issues that require action.
- ✅ Avoid alert fatigue by reducing noise.
- ✅ Include clear context and runbook links.
- ✅ Test your alerts regularly.
- ✅ Confirm that alerts resolve after the issue is fixed.
- ✅ Use alerts to improve reliability, not just to be notified.

Incident Lab: The App Is Up, but Orders Are Failing

1. Incident Scenario

- The API process is running and the application is reachable.
- Users can access the site, but orders are failing.
- The database container is not available or not responding.
- We investigate logs, health checks, container status, networking and recent changes.
- Objective: find the root cause and restore service without unnecessary restarts.

2. Architecture and Symptoms



Observed Symptoms

- Website loads and API is up (HTTP 200 on /health).
- Orders fail with 500 errors.
- Application logs show database connection errors.
- Database container is stopped or unreachable.
- Readiness check fails, but liveness check may still pass.

3. Verify Application and API Health

```
# Check application health endpoint
curl -I https://orderflow.com/health
HTTP/1.1 200 OK
Content-Type: application/json

# Application container is running
docker compose ps
NAME              IMAGE                          STATUS
orderflow-app     ghcr.io/veriqta/orderflow      Up 5 minutes
orderflow-db      postgres:15                    Exit 1 (2 minutes ago)
orderflow-nginx   nginx:latest                   Up 5 minutes
```

4. Investigate the Failing Orders

```
# Try to place an order
curl -X POST https://orderflow.com/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":1}'
HTTP/1.1 500 Internal Server Error
Content-Type: application/json

{"error":"Database connection failed"}
```

5. Check Application Logs

```
# View recent application logs
docker compose logs --tail 50 app
2026-10-30T14:22:31.123Z ERROR [orderflow]
DatabaseError: could not connect to server:
Connection refused (ECONNREFUSED) 172.20.0.5:5432
  at connect (/app/node_modules/pg/lib/client.
js:45)
  at process.processTicksAndRejections
(node:internal/process/task_queues:95:5)
request_id=f3c2a1d9 method=POST path=/api/orders
status=500 duration_ms=12
```

6. Check Database Container

```
# Check container status
docker compose ps  db
NAME          IMAGE         STATUS
orderflow-db  postgres:15   Exit 1 (2 minutes ago)

# View logs
docker compose logs db
2026-10-30 14:20:11.456 UTC [1] FATAL:
database files are incompatible with
server: data directory was initialized
by PostgreSQL 14, which is not compatible
with this version 15.
```

7. Additional Checks

```
# Check readiness endpoint
curl -I https://orderflow.com/ready
HTTP/1.1 503 Service Unavailable

# Test database connectivity from app
docker compose exec app \
  nc -zv db 5432
nc: connect to db (172.20.0.5) port 5432
failed: Connection refused

# Check Docker network
docker network inspect orderflow_default

# Review recent changes
git log --oneline -n 5
docker compose diff
Check for recent image or database version
changes
```

8. Diagnose the Root Cause

1. Application is running normally.
2. Orders fail due to database connection errors.
3. PostgreSQL container has stopped (Exit 1).
4. Logs show incompatible database version.
5. Recent change upgraded the Postgres image from 14 to 15 but used an old data directory.
6. The application cannot connect to the database.
7. Solution: use the correct compatible image or recreate the data directory with a migration plan.

9. Recover the Service

```
# Option 1: start the correct version (if data compatible)
docker compose up -d db

# Option 2: use the previous working image
docker compose pull postgres:14
docker compose up -d db

# Verify database is healthy
docker compose ps  db
docker compose exec db pg_isready

# Test order again
curl -X POST https://orderflow.com/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":1}'
```

10. Key Takeaways

- ☑ An application can be up but still fail to serve user requests.
- ☑ Always check logs, container status and health endpoints.
- ☑ Readiness checks detect dependency issues.
- ☑ Liveness checks only confirm the process is running.
- ☑ Investigate recent changes (image tags, configuration, database migrations).
- ☑ Do not restart everything immediately.
- ☑ Find the root cause and apply a targeted fix.
- ☑ Verify the service is fully restored with a test request.

Incident Lab: Disk Pressure, Timeouts, and a Slow Service

1. Incident Scenario

- The application is running but responses are slow and sometimes time out.
- Disk space is almost full.
- Containers may restart due to resource pressure.
- We investigate using system tools, Docker metrics and logs.
- Objective: identify the root cause, apply a safe fix, and verify improvement without losing evidence.

2. Common Causes

Disk exhaustion
Logs, data or temporary files consume all available space.

Resource contention
High CPU or memory usage causes slow responses.

Slow database or external service
Increased latency and timeouts.

Container restarts
Service restarts due to OOM or disk pressure.

Increased error rates
Requests fail or time out (5xx).

3. Symptoms to Look For

- Slow response times.
- HTTP 5xx errors and timeouts.
- High disk usage (near 100%).
- High CPU or memory usage.
- Containers restarting frequently.
- Application logs show errors (e.g. no space left on device).
- Users unable to complete orders or critical actions.

4. Check Disk Usage (df -h)

```
# Check disk space on the server
df -h

Filesystem      Size  Used Avail Use% Mounted on
/dev/root        50G   48G  1.2G  98% /
tmpfs           1.9G   12M  1.8G   1% /run
tmpfs           7.9G     0  7.6G   0% /dev/shm
/dev/sda1       200G  150G   42G  79% /var/lib/docker
```

5. Monitor Container Resources

```
# Check running containers and resource usage
docker stats

CONTAINER      CPU %   MEM USAGE / LIMIT   MEM %
orderflow-app  85.2%   1.8GiB / 2GiB       90%
postgres       12.4%   512MiB / 1GiB       50%
nginx          5.1%    64MiB / 256MiB      25%
```

6. Check Container Logs

```
# View recent application logs
docker compose logs --tail 50 app

2026-10-30T14:22:11.231Z  ERROR
Request timed out after 30s
  at database.query (db.js:45)
Error: connect ETIMEDOUT
2026-10-30T14:22:15.987Z  WARN
Retrying database connection...
2026-10-30T14:22:20.112Z  ERROR
write failed: no space left on device
```

7. Common Log Errors and Meanings

Log Message	Meaning
no space left on device	Disk is full (logs, data or temp files).
ETIMEDOUT	Service not responding in time (usually database or network).
connection refused	Service not running or not listening on the expected port.
OOMKilled	Container killed due to high memory usage.
database is locked	Database under heavy load or resource contention.

8. Investigate Further

1. Check disk usage by directory
   `sudo du -sh /var/* 2>/dev/null | sort -h`
2. Find large files
   `sudo find /var -type f -size +100M`
3. Check running processes
   `top or htop`
4. Check Docker container restarts
   `docker compose ps`
5. Inspect database logs
   `docker compose logs --tail 50 db`
6. Check network connectivity
   `docker compose exec app ping -c 3 db`

9. Apply a Safe Corrective Action

- [x] Identify and remove or rotate large log files.
- [x] Clean up unused Docker images and volumes (only after investigation).
- [x] Increase disk space or move data to a larger volume.
- [x] Restart only the affected service if required (after collecting evidence).
- [x] Verify that disk usage has improved.
- [x] Confirm that the application and database are working normally.
- [x] Run a test order to check response time.

10. Example: Clean Up Docker Resources (After Investigation)

```
# View disk usage by Docker
docker system df

# Remove unused images (safe)
docker image prune -f

# Remove unused volumes (only if safe)
docker volume prune -f

# Check disk space again
df -h
```

11. Verify Improvement

```
# Check disk space
df -h
# Check container status
docker compose ps

# Check response time
curl -o /dev/null -s -w "Time: %{time_total}s
HTTP: %{http_code}\n" https://orderflow.com/

# Run a test order
curl -X POST https://orderflow.com/api/orders \
  -H "Content-Type: application/json" \
  -d '{"product_id":1,"quantity":1}'
```

12. Key Takeaways

- [x] Disk pressure can cause slow responses, timeouts and container restarts.
- [x] Always investigate and collect evidence before deleting files or restarting services.
- [x] Use df -h, docker stats and logs to find the root cause.
- [x] Look for application, database and infrastructure issues.
- [x] Apply a safe and targeted fix.
- [x] Verify that the service has recovered and performance has improved.
- [x] Keep an eye on disk usage and set up alerts to prevent recurrence.

Back Up the Database and Prove You Can Restore It

1. What We Are Doing

- Create a consistent backup of the OrderFlow database.
- Store the backup off the application server.
- Encrypt the backup.
- Restore the backup into an isolated database.
- Verify that the original order records are intact.
- Understand RPO and RTO.
- Learn why an untested backup is not a verified recovery plan.

2. Backup Architecture (Overview)



- ✓ Take consistent backups (logical or physical).
- ✓ Store backups outside the application server.
- ✓ Encrypt backups at rest and in transit.
- ✓ Keep multiple backup versions (retention).
- ✓ Regularly test restoration.

3. Key Concepts

Backup consistency	A backup represents a valid and complete state of the database.
Retention	How long backups are kept (e.g. 7 days, 30 days, 1 year).
Off-server storage	Store backups outside the main server (e.g. cloud storage) to survive server failure.
Encryption	Protect backup files from unauthorized access.
RPO (Recovery Point Objective)	Maximum acceptable amount of data loss (e.g. 15 minutes).
RTO (Recovery Time Objective)	Maximum acceptable time to restore the service (e.g. 1 hour).

4. Create a Database Backup

```
# Create a backup of OrderFlow database
docker exec orderflow-db pg_dump -U postgres \
    -d orderflow -F c -b -v \
    -f /backups/orderflow_$(date +%F).dump

# Compress and encrypt the backup
gzip /backups/orderflow_2026-10-30.dump
gpg -c orderflow_2026-10-30.dump.gz

# List backup file
ls -lh /backups
-rw-r--r-- 1 root root  12M Oct 30 14:22
orderflow_2026-10-30.dump.gz.gpg
```

5. Store Backup Off-Server

```
# Copy encrypted backup to cloud storage
aws s3 cp orderflow_2026-10-30.dump.gz.gpg \
    s3://veriqta-backups/orderflow/

# Verify the upload
aws s3 ls s3://veriqta-backups/orderflow/
2026-10-30 14:25:10    12582912
orderflow_2026-10-30.dump.gz.gpg
```

Off-Server Storage Options



6. Restore to an Isolated Database

```
# Create a new database (restore target)
docker exec -it orderflow-db psql -U postgres
CREATE DATABASE orderflow_restore;
\q

# Decrypt and restore
gpg -d orderflow_2026-10-30.dump.gz.gpg | \
 gunzip | docker exec -i orderflow-db \
  pg_restore -U postgres -d orderflow_restore \
  --clean --if-exists

# Check restored tables
docker exec -it orderflow-db psql -U postgres \
    -d orderflow_restore -c "\dt"
```

7. Verify the Restored Data

```
# Count orders in original database
docker exec -it orderflow-db psql -U postgres \
  -d orderflow -c "SELECT COUNT(*) FROM orders;"
# Count orders in restored database
docker exec -it orderflow-db psql -U postgres \
  -d orderflow_restore -c "SELECT COUNT(*) FROM orders;"

# Compare sample records
docker exec -it orderflow-db psql -U postgres \
  -d orderflow_restore -c \
   "SELECT id, product_id, quantity, created_at \
    FROM orders ORDER BY id DESC LIMIT 5;"
```

8. Example Output

Original Database

count
1250

Restored Database

count
1250

✓ The order records are intact and match the original database.

9. Backup and Recovery Checklist

- ✓ Backups are consistent and complete.
- ✓ Backups are stored off the application server.
- ✓ Backups are encrypted.
- ✓ Retention policy is defined and followed.
- ✓ Restoration is tested regularly.
- ✓ RPO and RTO are documented.
- ✓ We can restore the database into a clean environment.
- ✓ Data integrity is verified after restoration.
- ✓ Access to backups is restricted.
- ✓ Monitoring and alerts are in place for backup failures.

10. Why Untested Backups Are Not Enough

- A backup that has never been restored may be corrupted.
- You may not know the exact steps or required tools.
- Permissions, encryption keys or dependencies may be missing.
- Restoration may take longer than expected.
- You may discover data issues only during a real incident.
- Regular restore tests prove that your backups actually work.

11. Key Takeaways

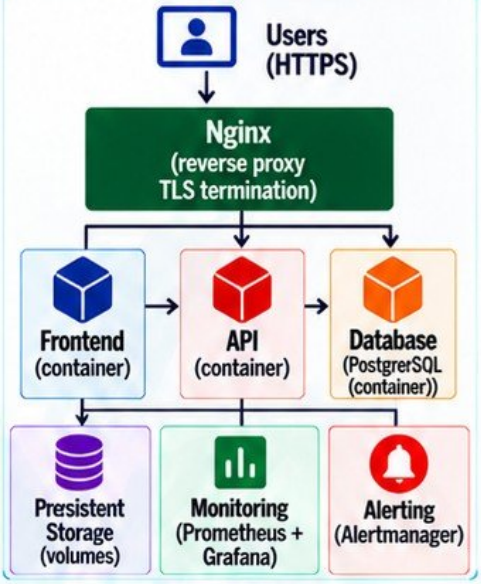
- ✓ Back up your database regularly and store it off-server.
- ✓ Encrypt backups and control access.
- ✓ Understand and document your RPO and RTO.
- ✓ Always test restoration in an isolated environment.
- ✓ Verified backups give you confidence during real incidents.

Production Readiness: Defend Your Deployment

1. Final Engineering Assessment

1. **Deployment:** containerized and versioned.
2. **HTTPS:** TLS enabled with valid certificate.
3. **Access control:** least privilege and secure access.
4. **Secrets:** not in code, stored securely.
5. **Persistence:** database data is durable and backed up.
6. **CI/CD:** automated build, test and deploy.
7. **Monitoring:** metrics and dashboards in place.
8. **Alerting:** critical alerts configured.
9. **Rollback:** previous version available and tested.
10. **Recovery:** database backup and restore tested.

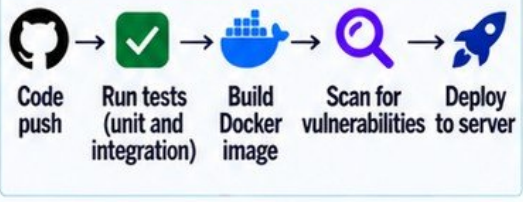
2. Production Architecture (Single Server)



3. Security and Access Control

- ☑ Enforce HTTPS (TLS).
- ☑ Use strong passwords and key-based access (no password SSH).
- ☑ Apply firewall rules (allow only required ports).
- ☑ Use least privilege for all services.
- ☑ Store secrets in environment-specific configuration or a secrets manager.
- ☑ Keep the system and containers updated.
- ☑ Restrict database access to the application only.

4. CI/CD Pipeline (Overview)



5. Monitoring and Alerting



- ☑ Track request rate, error rate, latency.
- ☑ Monitor CPU, memory, disk and container health.
- ☑ Set alerts for service downtime, high errors and low disk space.

6. Rollback and Recovery

Rollback
Redeploy previous image tag or digest.

Recovery
Restore database from a verified backup.

7. Demonstrate the Working Application

```
# Check application is running
curl -I https://orderflow.com/health
HTTP/1.1 200 OK
# View application in browser
https://orderflow.com
# Check running containers
docker compose ps
NAME            IMAGE                    STATUS        PORTS
orderflow-app   ghcr.io/veriqta/app:1.0.0  Up 5 minutes  0.0.0.0:8000->8000/tcp
orderflow-db    postgres:15              Up 5 minutes  5432/tcp
orderflow-nginx nginx:latest             Up 5 minutes  0.0.0.0:80->80/tcp
```

8. Recover from an Injected Failure

```
# Example: stop the database (simulate failure)
docker compose stop db
# Check application error
curl -I https://orderflow.com/api/orders
HTTP/1.1 500 Internal Server Error
# Investigate and recover
docker compose start db
# Verify recovery
curl -I https://orderflow.com/api/orders
HTTP/1.1 200 OK
```

9. Operational Handover (Example)

Deployment Commands
- git pull
- docker compose pull
- docker compose up -d

Verification Evidence
- Health endpoint: 200 OK
- Containers running (docker compose ps)
- API and frontend accessible via HTTPS
- Recent successful CI/CD run
- Monitoring dashboards working

Known Risks
- Single server (no high availability)
- Single point of failure
- Limited horizontal scalability
- Manual recovery steps may be required in some scenarios

10. Limitations of a Single-Server Architecture

- Single point of failure (server, network or disk).
- Limited scalability (vertical scaling only).
- No automatic failover or high availability.
- Maintenance can cause downtime.
- Resource contention between services.
- Higher risk during traffic spikes.
- Requires careful monitoring, backups and recovery planning.

11. Final Readiness Checklist

- ☑ Application deployed and accessible over HTTPS.
- ☑ Access control and firewall configured.
- ☑ Secrets are secure and not in code.
- ☑ Database persists data and backup is verified.
- ☑ CI/CD pipeline is working with tests and scanning.
- ☑ Monitoring and alerting are active.
- ☑ Rollback tested and works.
- ☑ Recovery tested and documented.
- ☑ Known risks are documented and understood.

12. Key Takeaways

- ☑ Production readiness is about people, process and evidence.
- ☑ A working deployment must be secure, recoverable and observable.
- ☑ Test your recovery before you need it.
- ☑ A single-server setup is suitable for learning and small workloads, but it has clear limitations in reliability and scale.
- ☑ Document everything for smooth handover and future maintenance.